INTERNSHIP REPORT



C/C++ Development Internship

Organization
Auspify Technologies

Intern Name
Muhammad Ibrahim

ID
AT/INT/2034723

Position
C/C++ Development Internship

Internship duration
4 weeks

Projects covered
Tasks 3, 4, 5, and 6

A record of project implementation, testing, and learning

Contents

1. Introduction 2
2. Internship Objectives 2
3. Development Environment and Tools 2
4. Overview of Completed Projects 2
4.1 Task 3 — Student Management System 2
4.2 Task 4 — Library Management System 3
4.3 Task 5 — Banking Management System 3
4.4 Task 6 — Expression Evaluator 3
5. Testing and Validation 3
6. Challenges and How I Approached Them 4
7. Skills and Learning Outcomes 4
8. Work Summary Across the Internship 4
9. Conclusion 5
10. Project Repository and Supporting Materials 5
Acknowledgement 5

1. Introduction

This report summarizes my work and learning experience during my four-week C/C++ Development Internship with Auspify Technologies. During the internship, I selected Tasks 3, 4, 5, and 6 from the assigned task list and developed four C++ console applications: a Student Management System, a Library Management System, a Banking Management System, and an Expression Evaluator.

The work gave me an opportunity to apply core programming concepts to practical problems. I worked on organizing program logic, separating code into source and header files where appropriate, handling user input, working with text files for data storage, and testing program behavior through different inputs.

2. Internship Objectives

- Strengthen practical programming skills in C++.
- Apply functions, classes, objects, conditional statements, loops, and input validation in working applications.
- Practice organizing code across source and header files.
- Use text files to store and retrieve application data where required.
- Test normal use cases and consider invalid input and error conditions.
- Prepare clear project folders, screenshots, and documentation for final review.

3. Development Environment and Tools

The projects were developed as console applications using C++. The development environment and tools used were:

Programming language: C++

Code editor: Visual Studio Code

Compiler: GCC/MinGW (g++)

Data storage: Text files for projects that persist records

Version control: Git, with GitHub planned for repository hosting

The applications are designed to run in a terminal or command prompt. The focus was on program logic, reliability, and clear console interaction rather than a graphical user interface.

4. Overview of Completed Projects

4.1 Task 3 — Student Management System

The Student Management System was developed to manage student records through a console-based menu. It provides a practical example of storing and manipulating structured records in a small application.

The project includes operations for managing student information, such as adding records, displaying records, searching for a student, updating details, and deleting records, as implemented in the application. Student data is stored in a text file so that records can be retained between runs.

This task helped me practice designing menu-driven programs, organizing record-handling logic, and keeping data operations separate from the main interaction flow.

4.2 Task 4 — Library Management System

The Library Management System was developed to organize book records and support common library operations through a console interface.

The application supports book-related operations such as adding, removing, searching, and displaying books. It also includes issue and return workflows where supported by the implementation. Book records are stored in a text file for persistence.

This task strengthened my understanding of managing collections of records, checking the state of a record before performing an operation, and designing clear menu options for users.

4.3 Task 5 — Banking Management System

The Banking Management System simulates basic account-management operations in a console application. It was developed as a learning project and is not intended for real financial use.

The project includes account-related operations such as creating and managing accounts, viewing balances, depositing and withdrawing funds, and saving account records to a text file, according to the implemented features. Input validation is important in this project to help prevent invalid amounts and unsupported operations.

This task provided practice in applying validation to operations that affect stored values and in keeping related account behavior organized across program files.

4.4 Task 6 — Expression Evaluator

The Expression Evaluator calculates arithmetic expressions entered by the user. It was an opportunity to work with parsing and evaluating input rather than relying only on straightforward menu-based record management.

The evaluator is intended to handle arithmetic operators, operator precedence, parentheses, and supported mixed bracket types. Decimal values and invalid expressions are also useful test cases. Division by zero and malformed expressions should be handled safely according to the program's implemented error checks.

This task helped me think more carefully about the order of operations, nested expressions, input validation, and the importance of testing edge cases. For example, the expression [4 + {5 - (6 * 8)}] / 9 evaluates to -25/9, approximately -2.7778.

5. Testing and Validation

After development, I ran the applications and checked representative workflows. I also prepared sample data and captured screenshots to document the visible behavior of each program.

Student Management System: Checked record display and common student-record operations.

Library Management System: Checked book records and available library operations.

Banking Management System: Checked account information and transaction-related operations using fictional test data.

Expression Evaluator: Checked arithmetic calculations, operator precedence, nested brackets, decimal values, and error cases supported by the implementation.

Testing was performed manually through the console. The sample data used for demonstration should remain fictional and must not contain real personal or financial information.

6. Challenges and How I Approached Them

One of the main challenges in console applications is ensuring that input is interpreted correctly and that each menu operation behaves as expected. I approached this by running the programs repeatedly with representative inputs and reviewing the results.

File-based storage also requires consistency between the format used when saving records and the format expected when reading them. Keeping the sample data aligned with the program's actual file format is important for reliable startup and testing.

For the Expression Evaluator, arithmetic precedence and nested brackets require careful handling. Testing expressions with different operators, brackets, and invalid input helped me identify cases that needed attention.

These projects reinforced that completing an application involves more than writing code: it also includes testing, organizing files, preparing evidence of functionality, and documenting how the work can be built and run.

7. Skills and Learning Outcomes

- Improved confidence in writing and debugging C++ console applications.
- Applied object-oriented programming and code modularization where appropriate.
- Practiced handling records and persisting data in text files.
- Gained experience with input validation and error handling.
- Improved my approach to testing normal and edge-case inputs.
- Learned to organize project folders, maintain a README, and use Git for version control.
- Practiced presenting technical work through screenshots and written documentation.

8. Work Summary Across the Internship

The internship lasted four weeks. The work progressed from understanding the assigned tasks and setting up the development environment to implementing the selected projects, testing their behavior, preparing sample data, and organizing the final project materials.

The summary below describes the overall stages of work. It is not a day-by-day log; the precise order and timing of individual activities may vary depending on how the work was completed.

Planning and setup: Reviewed the assigned task list, selected Tasks 3–6, and prepared the C++ development environment.

Implementation: Developed the Student Management, Library Management, Banking Management, and Expression Evaluator projects.

Testing and refinement: Ran the applications, checked representative operations, reviewed input and error cases, and prepared fictional sample data.

Documentation and finalization: Captured screenshots, organized source files, and prepared the project report, presentation, and repository for final review.

9. Conclusion

The four-week C/C++ Development Internship gave me practical experience applying programming fundamentals to several console-based applications. By completing the four selected tasks, I worked with record management, text-file persistence, input validation, arithmetic evaluation, and manual testing.

The experience also highlighted the importance of presenting work clearly. Source code, sample data, screenshots, build instructions, and concise documentation all contribute to making a project easier to understand and review.

I intend to use the skills developed during this internship as a foundation for further work in C++, software design, debugging, and larger application development.

10. Project Repository and Supporting Materials

The project source code, sample data, and screenshots are organized in separate folders for each task. The README provides an overview of the projects and instructions for compiling and running them.

GitHub repository:

Acknowledgement

I would like to thank Auspify Technologies for the opportunity to participate in this internship and work on practical C++ programming tasks. The projects provided a useful opportunity to apply programming concepts, strengthen my problem-solving approach, and reflect on the importance of testing and documentation.